

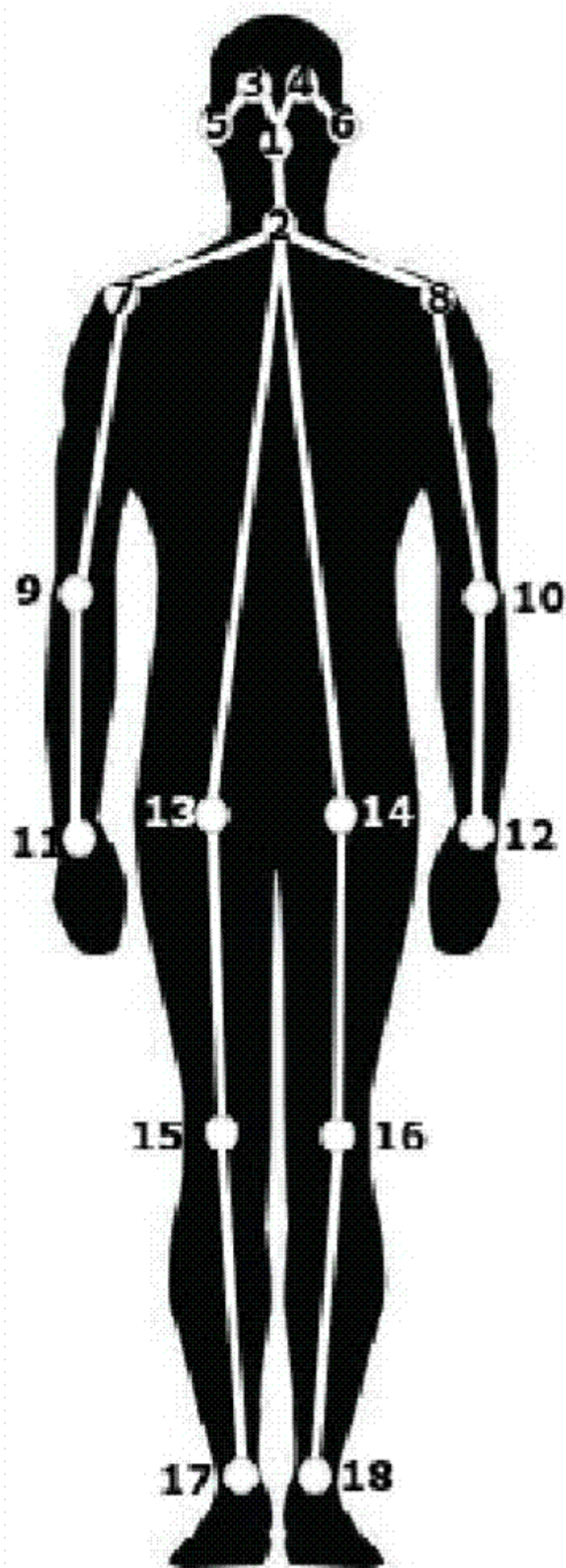
13.2 Human Pose Estimation

Source Code Path:

```
/home/yahboom/dofbot_ws/src/yahboom_vision/yahboom_vision/detect_pose.py
```

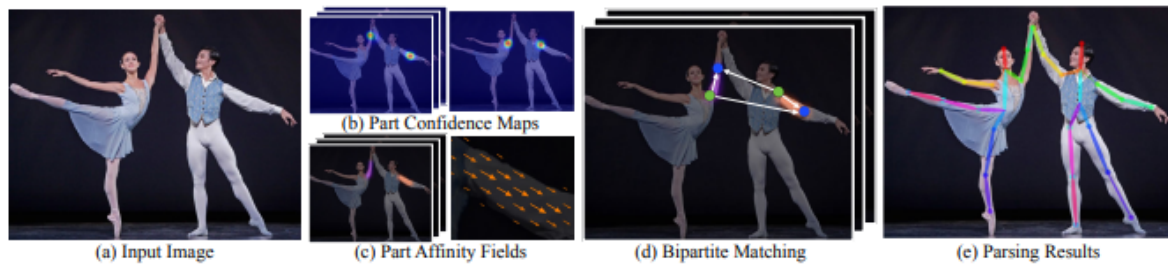
5.3.1 Overview

Human Pose Estimation involves estimating the posture of a human body by correctly connecting the keypoints detected within an image. These keypoints typically correspond to joints on the human body that possess a certain degree of freedom—such as the neck, shoulders, elbows, wrists, waist, knees, and ankles—as illustrated in the figure below.



- 1 鼻子
- 2 脖子
- 3 左眼
- 4 右眼
- 5 左耳
- 6 右耳
- 7 左肩
- 8 右肩
- 9 左肘
- 10 右肘
- 11 左腕
- 12 右腕
- 13 左髓
- 14 右髓
- 15 左膝
- 16 右膝
- 17 左踝
- 18 右踝

5.3.2 Principles



An input image is fed into a convolutional network to extract features, yielding a set of feature maps. These maps are then split into two branches, where separate CNN networks are employed to extract "Part Confidence Maps" and "Part Affinity Fields" (PAFs), respectively.

Once this information is obtained, we utilize "Bipartite Matching" (a concept from graph theory) to determine "Part Associations"—that is, to link the specific joints belonging to the same individual. Due to the inherent vectorial nature of PAFs, the resulting bipartite matches are highly accurate, ultimately allowing us to assemble the complete skeletal structure for each person.

Finally, based on the PAFs, we perform "Multi-Person Parsing." This involves transforming the multi-person parsing problem into a graph-based problem, which is then solved using the Hungarian Algorithm.

(The Hungarian Algorithm is the most common algorithm for bipartite matching. Its core principle lies in finding "augmenting paths," making it an effective method for determining the maximum matching in a bipartite graph.)

5.3.3 Launching

```
# Launch the camera
ros2 launch dofbot_info camera_arm_kin.launch.py
# Launch the pose estimation program
ros2 run yahboom_vision detect_pose
```

After clicking on the image window, press the [f] key on your keyboard to toggle the object detection feature. **

```
if action == ord('f') or action == ord('F'): state = not state # Toggle function
```

Input Image



Output Image

84.90ms



You can see the actual footage captured by the camera.

frame

FPS : 7

Openpose

(x=462, y=425) ~ R:253 G:251 B:255

